

Generated Drifts

X and a hand-designed
Randers field ω
`run_swiss_roll_generated.py /`
`run_mammoth_generated.py:`
`make.*randers()`

Locate: $X_{aug} = [X; X + \omega]$,
plain Euclidean geodesic
 $D_{sym,aug}$ over all $2n$
points (knn/Dijkstra,
 ω NOT injected here)
`randers.bridge.py:`
`fdgl_pipeline()` L271--278

(real,real) block swapped:
 $D_{asym} = \text{compute_dist_matrix}(X, \omega)$
(directed Dijkstra), then
 $D_{sym,aug}[n, :n] =$
 $0.5(D_{asym} + D_{asym}^T)$
`randers.bridge.py:`
`fdgl_pipeline()` L280--293

`classical_mds( $D_{sym,aug}, d$ )` on
all $2n$ points – one determin-
istic placement, no training
`randers.bridge.py:`
`fdgl_pipeline()` L295--299

split $Y_{aug0} \rightarrow Y_{real0}, Y_{virtual0}$
 $B_{located} = Y_{virtual0} -$
 Y_{real0} , clipped, FROZEN
 $Y_{init} = Y_{real0}$
`randers.bridge.py:`
`fdgl_pipeline()` L300--305

Apply:
 $D_{asym} = \text{compute_dist_matrix}(X, \omega)$
(directed, dense);
 $D_{geo} = \text{compute_dist_matrix}(X, \text{None})$
(drift-free counterpart)
`randers.bridge.py:`
`fdgl_pipeline()` L322--330

`fdgl_low_dim()` – **Internal Pipeline**
(shared code, runs identically for both families from here on)

build directed k -NN membership: $A, \text{knn_mask} = \text{knn_weights}(D_{asym}, k)$
`randers_fdgl.py:` L367, `knn_weights()` L93

A_{geo} : Euclidean-weight counterpart, only needed if
`randers_attractive/randers_repulsive=False` – $A_{geo} = \text{knn_weights}(D_{geo}, k)$ if
 D_{geo} given, else falls back to A (warns: drift may leak into the “Euclidean” term)
`randers_fdgl.py:` L369--383

$(a, b_{param}) = \text{find_ab_params}(\text{spread}, \text{min_dist})$; $N =$
 $\text{compute_N}(D_{asym})$ (once, fixed for the whole run)
`randers_fdgl.py:` L385--386, `find_ab_params()` L125, `compute_N()` L185

if `force_model` = “fr_gravity”: $A_{edges} = \text{knn_mask}$
 $\vee \text{knn_mask}^T$ (undirected union); $k_{nat} = \text{fr_k}$ or $1/\sqrt{n}$
`randers_fdgl.py:` L390--395

$Y = Y_{init_override}$ if given, else `classical_mds( $D_{asym}, d$ )`; $Y_{init} = Y.\text{copy}()$
`randers_fdgl.py:` L397--401, `classical_mds()` L144

$B = \mathbf{0}$ if B_{fixed} is None else $B_{fixed}.\text{copy}()$; $M = \text{node_mass}$ or all-ones; if B_{fixed}
given and `scale_B.fixed`: precompute its unit direction once
`randers_fdgl.py:` L403--413

snapshot pre-capture (epoch-0 B , if `snapshot_every` given); verbose config print
`randers_fdgl.py:` L415--456

----- EPOCH LOOP: repeats n_epochs times -----

ramp schedule: $s = 0$ (first 30% of epochs) $\rightarrow$ linear 0 $\rightarrow 1$
(30–70%) $\rightarrow s = 1$ (last 30%); $s = 1$ always if `ramp=False`
`randers_fdgl.py:` L461--465

recompute pairwise geometry from the CURRENT Y : diff, d_{mat} , unit vectors e
`randers_fdgl.py:` L467--469

B_{fixed} given $\rightarrow B = s \cdot B_{fixed}$ (frozen direction;
optional live k -NN-distance magnitude rescale)
`randers_fdgl.py:` L471--483

`use_drift=True`, $B_{fixed} = \text{None}$ $\rightarrow B =$
 $s \cdot \text{compute_drift}(N, \text{knn_mask}, k, Y, \text{clip_delta})$
`randers_fdgl.py:` L484--486, `compute_drift()` L201

build (ρ_r, g_r) Randers-perturbed and (ρ_e, g_e) Euclidean pairs; select
 $\rho_{attr}/g_{attr}, \rho_{rep}/g_{rep}$ per `randers_attractive/randers_repulsive`
`randers_fdgl.py:` L488--495

`force_model`=“umap”: UMAP (a, b) -curve
 $\text{attr_coeff}/\text{rep_coeff}$, weighted by A_{attr}/A_{rep}
`randers_fdgl.py:` L499--512

`force_model`=“fr_gravity”: Bannister et al.: $f_a = \rho/k_{nat}$ (edges only,
 A_{edges}), $f_r = k_{nat}^2/\rho^2$ (all pairs)
`randers_fdgl.py:` L513--523

`step` = `force.sum(axis=1)` – net force on each y_i
`randers_fdgl.py:` L524--529

OPTIONAL, additive: if `negative_sampling`: draw S fresh random negatives
per node this epoch, replace/augment repulsion (formula differs by `force_model`)
`randers_fdgl.py:` L533--558

OPTIONAL, additive: if `use_gravity`: `step` += $\gamma M w_{grav} \cdot \text{gravity_term}$
(pulls $y_i \rightarrow y_i + b_i$; plain linear or Randers pseudo-edge curve)
`randers_fdgl.py:` L560--590

OPTIONAL, additive: if `use_virtual_neighbor`: `step`
+= $\text{attr_coeff}_v \cdot B$ (unconditional $(k+1)$ -th neighbour pull)
`randers_fdgl.py:` L593--597

clip `step` to `[-grad_clip, grad_clip]`; decaying lr $\text{cur_lr} = \text{lr} \cdot (1 -$
 $\text{epoch}/n_epochs)$; $Y += \text{cur_lr} \cdot \text{step}$; snapshot capture if `snapshot_every`
`randers_fdgl.py:` L599--606

----- end of epoch loop, next epoch (or exit after n_epochs) -----

final B consistency recompute: live drift recomputed once more against the
FINAL Y ; or B_{fixed} magnitude rescaled against the final Y ’s k -NN distance
`randers_fdgl.py:` L616--628

return $\{Y, Y_{init}, B, \text{mu}, \text{mu_directed}, \text{knn_mask}, N, a, b_{param}, \text{history}\}$
`randers_fdgl.py:` L633--641

Calculated Drifts

X alone – no field
injected anywhere

each point’s own true
directed k nearest neigh-
bors only $\rightarrow$ SPARSE
 D_{asym} (rest inf, NOT
Dijkstra-completed)
`isumap_bridge.py:`
`build_isumap_dist_matrix()` L26

D_{geo} derived ALGE-
BRAICALLY from D_{asym}
itself: average if both dir.
finite, else the finite one
`run_mammoth_calculated.py`
etc.: L130--132

Dijkstra-COMPLETE a
copy of D_{asym} (diameter-
fill unreachable pairs) $\rightarrow$
`classical_mds` $\rightarrow Y_{init}$
`isumap_bridge.py:`
`isumap_style_init()` L49

$N = \text{compute_N}(D_{asym})$;
if `--fixed-drift`:
 $B_{fixed} = \text{compute_drift}(N, \text{knn_mask}, k, Y_{init})$
once, FROZEN; else
 $B_{fixed} = \text{None}$ (de-
rived live each epoch)

`randers_fdgl.py:`
`compute_N()` L185;
`run_mammoth_calculated.py:`
L146--156